ADVANCED WEB TECHNOLOGIES · PROJECT REVIEW

EcoTrace

Code structure: which file does what

Every file in the project, grouped into frontend, backend, machine-learning service and configuration, with the language it is written in, what it does, and how the files work together.

Team member	Registration No.
Mohammed Afaan Sherief	24MID0178
Amit Kumar	24MID0155
Bhumi Kumari	24MID0216

1. The big picture

EcoTrace is split into three folders, one per running program, plus a few files at the top level. About 6,100 lines of code in total.

Folder	Part	Main language	Runs on	Role
`client/`	Frontend	JavaScript (React, JSX), CSS, HTML	The user's browser	Every screen the user sees; talks only to the API
`server/`	Backend (API)	JavaScript (Node.js, Express)	Server, port 5050	Login, data, CO_2 calculation, analytics, reports, alerts, live updates; also serves the built website
`ml/`	Backend (ML microservice)	Python (FastAPI)	Server, port 8000	Forecasting, delay risk, explanations, anomalies, clustering, optimizer
(no folder)	Database	MongoDB documents (JSON-like)	Port 27018 locally, or Atlas	Stores everything; its structure is defined in `server/src/models/index.js`

Folder tree

```
web proj/
├── client/                  FRONTEND (React)
│   ├── index.html           the single HTML page
│   ├── vite.config.js       dev server and build settings
│   ├── public/favicon.svg   browser-tab icon
│   └── src/
│       ├── main.jsx         starts React
│       ├── App.jsx          list of pages (routes)
│       ├── api.js           how the browser calls the API
│       ├── styles.css       all styling, light and dark theme
│       ├── context/         shared login state, live updates
│       ├── components/      reusable pieces (layout, map, charts)
│       └── pages/           one file per screen (13 screens)
├── server/                  BACKEND API (Node.js + Express)
│   └── src/
│       ├── index.js         starts the server
│       ├── config.js, db.js  settings and database connection
│       ├── models/          database schemas (Mongoose)
│       ├── middleware/      login and role checks
│       ├── routes/          API endpoints (URLs)
│       ├── services/        the business logic
│       ├── jobs/            scheduled tasks (cron)
│       └── seed.js          creates the demo company
├── ml/                      BACKEND ML SERVICE (Python)
│   ├── app/                 FastAPI app and the 6 ML components
│   └── scripts/             data generator and evaluation
├── start-local.sh           run everything on this Mac
├── start-public.sh          run everything + public link
├── render.yaml              cloud deployment recipe
└── README.md                setup instructions
```

Languages used

Language	Used for	Files
JavaScript (ES6+)	All backend API code (Node.js), frontend logic, build config	`server/src/**/*.js`, `client/src/api.js`, `client/src/main.jsx`, `client/vite.config.js`
JSX (JavaScript + HTML-like tags)	React components: every page and UI piece	`client/src/**/*.jsx`
CSS3	Layout, colours, light/dark theme, responsive design	`client/src/styles.css`
HTML5	The one page React mounts into	`client/index.html`
Python 3	Machine learning, optimization, synthetic data	`ml/app/*.py`, `ml/scripts/*.py`
JSON	Package lists and dependency versions	`package.json`, `package-lock.json`
YAML	Cloud deployment recipe	`render.yaml`
Bash (shell script)	One-command start scripts	`start-local.sh`, `start-public.sh`
Markdown	Setup documentation	`README.md`
SVG (XML)	Icon	`client/public/favicon.svg`
MongoDB query language	Aggregation pipelines written inside JavaScript	`server/src/services/analytics.js`, routes

2. Frontend: the client/ folder

Built with **React 18** and **Vite**. React builds the page out of components; Vite runs a fast development server (port 5173) and bundles the code into plain HTML, CSS and JavaScript for production (`client/dist`, which the API then serves). The browser never talks to the database; every piece of data comes from the API through `api.js`.

Entry point and configuration

File	Language	What it does	Lines
`index.html`	HTML	The only HTML page. Contains an empty `<div id="root">` that React fills, plus the title and favicon.	14
`vite.config.js`	JavaScript	Vite settings: React plugin, dev port 5173, and a proxy that forwards `/api` and `/socket.io` to the API on port 5050.	13
`package.json`	JSON	Lists the frontend libraries (React, React Router, Axios, Recharts, Leaflet, React Flow, Socket.io client) and the `dev` / `build` commands.	-
`public/favicon.svg`	SVG	The green leaf icon in the browser tab.	-
`src/main.jsx`	JSX	Starting point. Loads the CSS (app, Leaflet maps, React Flow) and renders `<App>` inside the router and the login provider.	18
`src/App.jsx`	JSX	The route table: which URL shows which page. Public pages (`/`, `/login`, `/register`) and protected `/app/...` pages that need a login.	47
`src/api.js`	JavaScript	Axios client for all API calls. Adds the login token to every request, and on a 401 response signs the user out.	22
`src/styles.css`	CSS	All styling: colour variables for light and dark themes, sidebar layout, cards, tables, buttons, badges, the landing page, and mobile layouts.	273

Shared state and reusable components

File	Language	What it does	Lines
`src/context/AuthContext.jsx`	JSX	React Context shared by every page: the signed-in user and company, login / register / logout, role checks, theme (light/dark), toast messages, the alert list, and the Socket.io connection that triggers live refreshes.	113
`src/components/Layout.jsx`	JSX	The app frame: sidebar navigation, top bar with company name, theme toggle and alert bell with dropdown.	89
`src/components/ui.jsx`	JSX	Small reusable pieces and helpers: number/₹/kg/date formatters, KPI card, mode chip, risk badge, budget bar, modal, loading and error boxes, chart tooltip and legend, icons, `useApi` data hook, theme-aware chart colours.	191
`src/components/RouteMap.jsx`	JSX	Leaflet map of India with each route drawn as a line whose width and shade grow with its CO_2.	50
`src/components/ForecastChart.jsx`	JSX	Recharts chart of actual monthly CO_2 plus the dashed 3-month forecast and its confidence band.	32

Pages (one file per screen)

File	Language	What it does	Lines
`src/pages/Landing.jsx`	JSX	Public home page with the whole project write-up: problem, objectives, workflow, modules, ML, architecture diagram, tech stack, feasibility, team.	407
`src/pages/Auth.jsx`	JSX	Login page (with one-click demo accounts) and Register page (creates a company and owner).	116
`src/pages/Dashboard.jsx`	JSX	KPI cards, monthly CO_2 by mode, budget bar, mode shares, forecast, top vendors, route map, high-risk shipments, top routes.	169
`src/pages/Shipments.jsx`	JSX	Shipments table with filters and paging; “Log shipment” form with live CO_2 preview; CSV import; mark delivered; risk explanation pop-up.	346
`src/pages/Vendors.jsx`	JSX	Vendor table with CO_2, on-time rate and K-Means segment; add/edit/delete form; “Segment vendors” button.	131
`src/pages/Locations.jsx`	JSX	Factory, warehouses and customers; add/edit/delete; network map of all sites.	117
`src/pages/Compare.jsx`	JSX	Green option comparator: destination, material, weight, priority sliders, ranked options and recommendation.	142
`src/pages/Optimizer.jsx`	JSX	Editable demand table, CO_2 budget slider, runs the optimizer and shows savings, shadow price and the allocation plan.	138
`src/pages/WhatIf.jsx`	JSX	What-if simulator: sliders that move freight between modes and show CO_2, cost and time changes instantly.	101
`src/pages/SupplyChain.jsx`	JSX	React Flow network of vendors → factory/warehouse → customers, links sized by CO_2, click to focus a node.	128
`src/pages/Insights.jsx`	JSX	ML insights: forecast and backtest, delay-model metrics, vendor segments, anomaly list with scan and “Mark OK”.	192
`src/pages/Reports.jsx`	JSX	View, download or email the monthly PDF; carbon-budget status; alert history.	101
`src/pages/Settings.jsx`	JSX	Three tabs: company and budget, editable emission factors, team members and roles.	181

3. Backend API: the server/ folder

Written in **JavaScript on Node.js** with the **Express** framework. It follows a layered structure, so each kind of work lives in its own folder:

- **routes/** receive HTTP requests and send JSON responses (the “what URL does what” layer);
- **services/** hold the actual logic (calculations, aggregations, PDF, email, ML calls), so routes stay short;
- **models/** describe how data is stored in MongoDB;
- **middleware/** runs before routes to check the login and the role.

Start-up and configuration

File	Language	What it does	Lines
`src/index.js`	JavaScript	Starts everything: Express app, JSON parsing, CORS, mounts every route, error handler, serves the built website from `client/dist`, sets up Socket.io with login check, connects the database, loads emission factors, seeds the demo company if empty, starts cron jobs, listens on port 5050.	71
`src/config.js`	JavaScript	Reads settings from `.env` (port, database URI, JWT secret, ML URL and key, OpenRouteService key, SMTP email) with safe	20
`src/db.js`	JavaScript	Connects to MongoDB: the `MONGO_URI` (e.g. Atlas) if set, otherwise reuses or starts a local MongoDB that saves data in `server/.data`.	39
`package.json`	JSON	Backend libraries (express, mongoose, jsonwebtoken, bcryptjs, multer, csv-parser, nodemailer, node-cron, pdfkit, socket.io, axios, dotenv) and the `start` / `dev` / `seed` commands.	-
`.env.example`	Text	Template for the secret settings file `.env` (which is never committed).	-

Database models and security

File	Language	What it does	Lines
`src/models/index.js`	JavaScript (Mongoose)	Defines the 9 MongoDB collections and their fields: Company, User, Location, Vendor, Shipment, EmissionFactor, Forecast, Alert, DistanceCache. Also indexes, including geospatial (2dsphere) indexes.	160
`src/middleware/auth.js`	JavaScript	Creates and verifies JWT login tokens; `requireAuth` blocks requests without a valid token; `requireRole` blocks actions the user’s role may not do; `asyncHandler` passes errors to the error handler.	39

Routes: the API endpoints

File	Language	What it does	Lines
`src/routes/auth.js`	JavaScript	`/api/auth`: register (company + owner + factory), login, current user, team list / add / remove. Hashes passwords with bcrypt.	64
`src/routes/directory.js`	JavaScript	`/api/vendors` and `/api/locations`: create, list, edit, delete (blocked if shipments exist), city geocoding, nearby-vendor geospatial search.	106
`src/routes/shipments.js`	JavaScript	`/api/shipments`: list with filters, live preview, create, mark delivered, delete, re-score risk, CSV import (Multer + csv-parser) and template download.	178
`src/routes/analytics.js`	JavaScript	`/api/analytics`: dashboard summary, supply-chain graph data, per-mode totals for the what-if simulator.	58
`src/routes/planning.js`	JavaScript	`/api/compare` (builds and scores every vendor × mode option) and `/api/optimize` (baseline from last month, builds LP options, calls the ML optimizer).	163
`src/routes/insights.js`	JavaScript	`/api/ml`: ML status, forecast (cached), metrics, retrain, anomaly scan and list, vendor clustering, explain one shipment's risk.	80
`src/routes/admin.js`	JavaScript	Reports (PDF view and email), alerts, company settings and budget, emission factors, team list.	67

Services: the business logic

File	Language	What it does	Lines
`src/services/emissions.js`	JavaScript	Emission factors per mode and the core formulas: CO_2 = distance × tonnes × factor, freight cost, transit days.	62
`src/services/routing.js`	JavaScript	Distances: OpenRouteService truck route (or straight-line × 1.25), per-mode distances for road/rail/sea/air, city geocoding, distance cache.	88
`src/services/cities.js`	JavaScript	Built-in table of about 45 Indian cities with coordinates and port flags, used when there is no OpenRouteService key.	58
`src/services/shipmentBuilder.js`	JavaScript	Turns form or CSV input into a complete shipment: validates it, finds both places, picks the distance, calculates CO_2, cost and days.	73
`src/services/analytics.js`	JavaScript	MongoDB aggregation pipelines: one `$facet` query for totals, monthly, by mode, vendor and route; monthly CO_2 for budgets and forecasting.	149
`src/services/ml.js`	JavaScript	Client for the Python ML service (with the shared API key), plus fallbacks when it is offline: rule-based delay risk and a seasonal-naive forecast.	137
`src/services/alerts.js`	JavaScript	Creates alerts, pushes them live with Socket.io, and checks the carbon budget (80% and 100% thresholds), emailing owners.	55
`src/services/report.js`	JavaScript	Builds the monthly sustainability PDF with pdfkit: KPIs, budget bar, mode bars, vendor and route tables, recommendations.	136
`src/services/mailer.js`	JavaScript	Sends email with Nodemailer; without SMTP settings it only logs the message.	25

Background jobs and demo data

File	Language	What it does	Lines
`src/jobs/cron.js`	JavaScript	node-cron schedules: weekly model retraining (Sun 02:00), monthly PDF report email (1st, 08:00), daily budget check (09:00).	42
`src/seed.js`	JavaScript	Creates the demo company: 3 users, 11 vendors, 10 locations and about 4,900 synthetic shipments over 2 years with monsoon, festival and late-vendor patterns, plus 14 deliberate errors.	206
`src/seedCli.js`	JavaScript	Command-line wrapper so `npm run seed` resets the demo data.	8

4. Machine-learning service: the ml/ folder

Written in **Python** with **FastAPI**. It is part of the backend, a separate microservice, because the ML libraries are Python. Only the Node API calls it (never the browser), using a shared secret key. It reads training data straight from MongoDB.

File	Language	What it does	Lines
`app/main.py`	Python (FastAPI)	The ML web service: checks the API key, defines the endpoints (`/health`, `/train`, `/metrics`, `/forecast`, `/delay-risk`, `/anomalies`, `/vendor-clusters`, `/optimize`), and saves and loads trained models with joblib.	168
`app/data.py`	Python (pymongo, pandas)	Reads shipments, vendors and tariffs from MongoDB into pandas tables; adds features such as monsoon month, festival month and transport mode.	84
`app/forecast.py`	Python (statsmodels)	Monthly CO_2 forecasting: tries several SARIMA(X) models with Indian season effects and picks the best by backtest (Prophet if installed); returns 3 months with an 80% band and a comparison with the naive baseline.	142
`app/delay.py`	Python (scikit-learn, XGBoost, SHAP)	Delay-risk model: builds features, trains Random Forest and XGBoost, keeps the better one, compares with a baseline, predicts late probability, explains each prediction with SHAP.	224
`app/anomaly.py`	Python (scikit-learn)	Isolation Forest that flags unusual or wrongly entered shipments (weight, cost vs tariff, distance vs route) and writes a reason for each.	63
`app/clusters.py`	Python (scikit-learn)	K-Means grouping of vendors by carbon intensity, price and on-time rate, with readable labels such as “Green and Reliable”.	54
`app/optimizer.py`	Python (PuLP)	Linear program for the cheapest shipping mix within demand, vendor capacity and CO_2 budget; reports the shadow price, or the lowest achievable CO_2 when the budget is impossible.	55
`app/__init__.py`	Python	Empty file that marks `app/` as a Python package.	0
`scripts/generate_synthetic.py`	Python (pandas, NumPy)	Generates 2 years of synthetic shipments as a CSV in the import format.	84
`scripts/evaluate.py`	Python	Prints the success-criteria metrics (forecast vs naive, delay model vs baseline, anomalies found).	37
`requirements.txt`	Text	Python libraries with exact versions (FastAPI, pandas, scikit-learn, XGBoost, statsmodels, SHAP, PuLP, pymongo…).	-
`.env.example`	Text	Template for the ML settings: database URI and the shared API key.	-

5. Top-level, configuration and generated files

Files at the top of the project

File	Language	What it does
`README.md`	Markdown	How to install, run, seed and deploy the project; demo accounts; results.
`start-local.sh`	Bash	One command to build the site and start the ML service and API on this Mac, then open http://localhost:5050.
`start-public.sh`	Bash	Same as start-local, plus a Cloudflare tunnel that prints a public link for teammates.
`render.yaml`	YAML	Recipe for deploying both services to Render cloud hosting.
`.gitignore`	Text	Tells git which folders to leave out (dependencies, secrets, build output, local data).
`EcoTrace_Presentation.pptx / .pdf`	Document	The review presentation.
`EcoTrace_Website_Guide.docx / .pdf`	Document	How the website works, plus FAQ.

Folders created automatically (not part of the source code)

Folder or file	Created by	Contains
`client/node_modules/`, `server/node_modules/`	`npm install`	Downloaded JavaScript libraries
`ml/.venv/`	`python3 -m venv`	Python virtual environment with the ML libraries
`client/dist/`	`npm run build`	The production website (bundled HTML, CSS, JS) served by the API
`server/.data/`	The API on first run	The local MongoDB database files
`ml/models/`	The ML service	Saved trained models (.joblib)
`logs/`	The start scripts	Log output of each service
`server/.env`, `ml/.env`	You (copied from `.env.example`)	Secret settings: never shared or committed
`package-lock.json`	`npm install`	Exact versions of every installed library

6. How the files work together

Following one action from the click to the database and back shows which file is responsible for each step.

Example 1: logging a new shipment

1. The user fills the form in **`client/src/pages/Shipments.jsx`** and clicks Save.
2. **`client/src/api.js`** sends **`POST /api/shipments`** with the login token.
3. **`server/src/index.js`** passes it to **`middleware/auth.js`**, which checks the token and the role.
4. **`routes/shipments.js`** receives it and calls **`services/shipmentBuilder.js`**.
5. **`shipmentBuilder.js`** gets the distance from **`services/routing.js`** (using **`cities.js`** or OpenRouteService) and the CO_2, cost and days from **`services/emissions.js`**.
6. **`services/ml.js`** asks the Python service (**`ml/app/main.py`** → **`ml/app/delay.py`**) for the delay risk and SHAP reasons.
7. The shipment is saved through the **`Shipment`** model in **`models/index.js`** into MongoDB.
8. **`services/alerts.js`** checks the carbon budget and broadcasts a Socket.io event; **`client/src/context/AuthContext.jsx`** receives it on every teammate's screen and refreshes their pages.

Example 2: opening the dashboard

1. **`client/src/pages/Dashboard.jsx`** calls **`GET /api/analytics/summary`** and **`GET /api/ml/forecast`** through the **`useApi`** hook in **`components/ui.jsx`**.
2. **`routes/analytics.js`** calls **`services/analytics.js`**, which runs one MongoDB aggregation for all totals and breakdowns.
3. **`routes/insights.js`** returns the cached forecast, or asks **`ml/app/forecast.py`** for a new one.
4. The page draws the results with Recharts, **`components/ForecastChart.jsx`** and **`components/RouteMap.jsx`**.

Example 3: running the optimizer

1. **`client/src/pages/Optimizer.jsx`** loads **`GET /api/optimize/baseline`**, then sends **`POST /api/optimize`**.
2. **`routes/planning.js`** builds every vendor × mode option using **`routing.js`** and **`emissions.js`**.
3. **`services/ml.js`** sends them to **`ml/app/main.py`**, and **`ml/app/optimizer.py`** solves the linear program with PuLP.
4. The plan comes back to the page, which shows savings and the allocation table.

> Rule of thumb: **pages** (frontend) → **routes** (which URL) → **services** (the logic) → **models** (the database). Anything "intelligent" goes from **`services/ml.js`** to the Python files in **`ml/app/`**.

7. Quick finder: “where is…?”

Useful when a reviewer asks to see a specific piece of code.

If asked about…	Open this file
The CO_2 formula and emission factors	`server/src/services/emissions.js`
How distance is calculated	`server/src/services/routing.js`
How a shipment is built from the form or CSV	`server/src/services/shipmentBuilder.js`
CSV upload and parsing	`server/src/routes/shipments.js` (import route)
Database schemas / collections	`server/src/models/index.js`
Login, JWT and roles	`server/src/middleware/auth.js`, `server/src/routes/auth.js`
Password hashing	`server/src/routes/auth.js` (bcrypt)
Dashboard aggregation queries	`server/src/services/analytics.js`
Geospatial queries	`server/src/routes/directory.js` (nearby vendors), indexes in `models/index.js`
Comparator scoring	`server/src/routes/planning.js` (`score` function); live re-ranking in `client/src/pages/Compare.jsx`
Optimizer linear program	`ml/app/optimizer.py`
Forecast model	`ml/app/forecast.py`
Delay model and SHAP	`ml/app/delay.py`
Anomaly detection	`ml/app/anomaly.py`
Vendor clustering	`ml/app/clusters.py`
Synthetic data	`server/src/seed.js` (demo company), `ml/scripts/generate_synthetic.py` (CSV)
Budget alerts and emails	`server/src/services/alerts.js`, `server/src/services/mailer.js`
PDF report	`server/src/services/report.js`
Scheduled jobs	`server/src/jobs/cron.js`
Real-time updates	`server/src/index.js` (server side), `client/src/context/AuthContext.jsx` (browser side)
Page routing (URLs)	`client/src/App.jsx`
Styling and dark mode	`client/src/styles.css`
Route map	`client/src/components/RouteMap.jsx`
Supply-chain graph	`client/src/pages/SupplyChain.jsx`
ML fallbacks when the Python service is off	`server/src/services/ml.js`